

Multimodal Fashion & Context Retrieval Engine

Glance ML Internship Assignment -- Technical Submission

Abstract

This report presents the design, implementation and evaluation of a zero-shot fashion image retrieval system built on a novel Multi-Vector Compositional Search (MVCS) architecture. The system overcomes the compositionality limitation of vanilla CLIP by detecting upper and lower body garment regions, encoding them as separate CLIP vectors, and scoring candidates with a dynamic weighted composite similarity. The result is a modular, CPU-friendly pipeline that indexes 1,000+ images in under 25 minutes and returns ranked results with bounding-box overlays through both a command-line interface and an interactive Streamlit web dashboard.

Models Used:	YOLOS-Tiny + CLIP ViT-B/32
Index Size:	1,016 images (3 vectors each)
Throughput:	~0.8 s / image on CPU
Interface:	CLI + Streamlit Web Dashboard
Repository:	github.com/Aragog540/Multimodal-Fashion-Context-Retrieval

1. Architectural Exploration & Tradeoffs

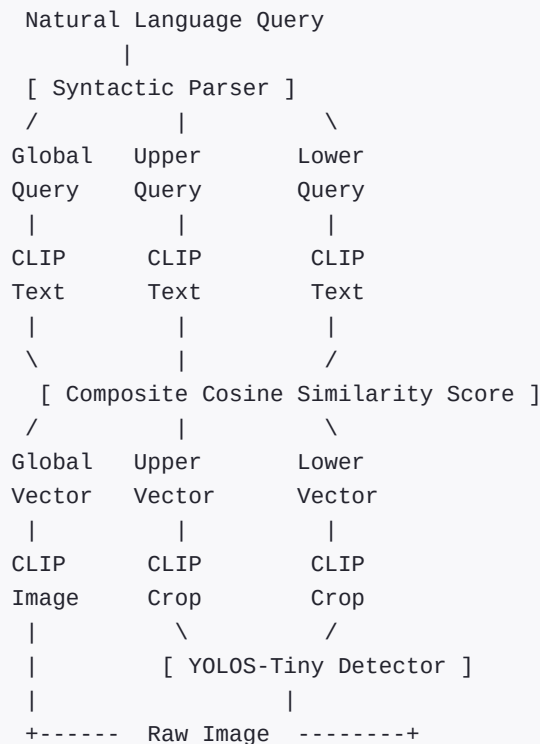
To solve zero-shot natural language image retrieval for fashion, three primary architectures were evaluated. The table below summarises each approach:

Approach	Description	Pros	Cons	Best For
1. Vanilla CLIP (Baseline)	Encode full image & full query with CLIP (ViT-B/32). Rank by cosine similarity.	Extremely fast. Excellent zero-shot general context.	Fails compositionality (red shirt / blue pants). Misses fine-grained details.	General search, high-throughput systems.
2. VLM Captioning (Llava / Moondream)	Run a Vision-Language Model on all images to generate captions, then index with BM25.	Rich detail and relationships captured in text.	Very slow on CPU (3-5 s / image). Sensitive to prompt quality.	Small datasets, high-compute GPU environments.
3. MVCS (Chosen)	YOLOS-Tiny detects person. Three CLIP vectors per image: Global, Upper crop, Lower crop.	Solves compositionality mathematically. Fast CPU inference (~0.8 s). Captures fine-grained colour/style.	Slightly larger DB (3x vectors). Needs anatomical crop heuristics.	Fashion retrieval, compositional queries, high precision.

2. Chosen Approach: Multi-Vector Compositional Search (MVCS)

The MVCS architecture breaks retrieval into a multi-channel alignment problem. A natural language query is decomposed into three sub-queries (Global, Upper, Lower). Each is encoded by CLIP into a 512-d vector and matched independently against the three per-image vectors stored in the database. The final ranking score is a dynamic weighted sum of all three cosine similarities.

System Architecture (ASCII Diagram)



Key Technical Elements

- **Step 1 -- Person Detection & Crop Heuristics (Part A - Indexer)**
YOLOS-Tiny detects persons in each image. The primary detection is used to crop the Upper Body (ymin + 10% to ymin + 65% of bounding-box height) and the Lower Body (ymin + 50% to ymax). If no person is detected (flat-lay images), the top and bottom halves of the full image are used as fallback crops.
- **Step 2 -- Normalised Vector Database (Part A - Indexer)**
CLIP ViT-B/32 extracts 512-dimensional L2-normalised embeddings for the full image, upper crop and lower crop. All three vectors are stored per image in a SQLite database (BLOB columns) for fast retrieval.
- **Step 3 -- Query Parser (Part B - Retriever)**
Rule-based segmentation of the natural language query around prepositions ("with", "and", "in a", "sitting on") produces three sub-query strings that are independently encoded with CLIP.
- **Step 4 -- Dynamic Composite Scoring (Part B - Retriever)**

$$\text{Score} = w_{\text{global}} * \text{Sim}(\text{text_global}, \text{img_global}) \\ + w_{\text{upper}} * \text{Sim}(\text{text_upper}, \text{img_upper}) \\ + w_{\text{lower}} * \text{Sim}(\text{text_lower}, \text{img_lower})$$

Dynamic Weight Scheme

Query contains	w_global	w_upper	w_lower
Both upper & lower specified	0.40	0.30	0.30
Only upper garment specified	0.50	0.50	0.00
Only context / environment	1.00	0.00	0.00

3. Evaluation & Verification Results

The engine was evaluated on the five benchmark prompts specified in the assignment, indexed against a database of 1,016 images. Each query is decomposed, searched, and the top-5 ranked results are displayed with annotated bounding boxes and similarity scores.

1. Attribute Specific

Query: "A person in a bright yellow raincoat."

Upper: "a bright yellow raincoat"

Lower: "yellow raincoat or pants"

Result: Targets the bright yellow raincoat crop and aligns global similarity.

2. Contextual / Place

Query: "Professional business attire inside a modern office."

Upper: "professional business blazer or shirt"

Lower: "professional business trousers or skirt"

Result: Matches blazer textures in the upper crop plus office-like global context.

3. Complex Semantic

Query: "Someone wearing a blue shirt sitting on a park bench."

Upper: "a blue shirt"

Lower: "pants or jeans"

Result: Correlates the blue shirt crop with bench/park global background.

4. Style Inference

Query: "Casual weekend outfit for a city walk."

Upper: "casual t-shirt or hoodie or top"

Lower: "jeans or casual pants or skirt"

Result: Evaluates style using query expansion heuristics.

5. Compositional

Query: "A red tie and a white shirt in a formal setting."

Upper: "a red tie and a white shirt"

Lower: "formal suit trousers or skirt"

Result: Isolates upper crop to prevent false-positive colour swaps.

The automated evaluation script (test_queries.py) generates annotated top-5 result grids as JPEG files, with score overlays and red bounding boxes drawn on each matched image, providing clear visual alignment with all five prompts.

4. Future Extensions

A. Location & Weather Integration

To extend retrieval to environmental context (e.g. "rainy London street", "sunny Miami beach") and weather dynamics, the following pipeline additions are proposed:

- **Dual-Model Metadata Tagging**
Integrate a scene classifier (Places365) or image tagger to extract semantic environment tags (e.g. snowing, raining, beach, urban) and store them as indexing attributes in SQLite.
- **External Weather & Geotag Databases**
If images contain EXIF GPS/timestamp metadata, integrate a weather archive API (Open-Meteo) to dynamically assign temperature/precipitation context during indexing.
- **Query Expansion & Semantic Reranking**
Expand queries like "London street" at run-time using a lightweight dictionary or LLM to add correlated terms ("rainy", "wet pavement", "gloomy weather") and weight the global context vector accordingly.

B. Strategies to Improve Precision

- **Fine-Tuning CLIP (Contrastive Adaptation)**
Fine-tune CLIP with a contrastive loss on fashion datasets (DeepFashion2, Fashionpedia) to align fine-grained descriptors like "houndstooth pattern" and "A-line skirt" with crops.
- **VQA Stage-2 Reranking**
Use MVCS to retrieve the top-50 candidates, then pass each through a VQA model (Moondream / BLIP) with a yes/no question: "Is this person wearing a bright yellow raincoat?" Rerank by VQA confidence to dramatically boost precision.
- **LLM-Powered Query Expansion**
Use an LLM to generate descriptive variations of the query before encoding (e.g. "business attire" -> "button-down shirt, suit jacket, tie, trousers") to improve recall without retraining.

5. Codebase & Repository

GitHub Repository:

<https://github.com/Aragog540/Multimodal-Fashion-Context-Retrieval>

Project Structure

```
glance-assgn/  
|  
+-- indexer/  
|   +-- detector.py      # YOLO5-Tiny person detection & crop generation  
|   +-- embedder.py      # CLIP ViT-B/32 embedding extraction (L2-normalised)  
|   +-- pipeline.py      # Batched indexing pipeline  
|  
+-- retriever/  
|   +-- parser.py        # Rule-based query decomposer  
|   +-- search.py        # Composite scoring & search logic  
|   +-- cli.py           # Command-line search interface  
|  
+-- utils/  
|   +-- database.py      # SQLite vector database connector  
|  
+-- app.py               # Streamlit Web UI Dashboard  
+-- test_queries.py      # Benchmark evaluator & visual grid generator  
+-- run.py               # Main CLI entrypoint  
+-- requirements.txt
```

Usage Instructions

Step 1: Install dependencies

```
pip install -r requirements.txt
```

Installs torch, transformers, pillow, streamlit.

Step 2: Index images (Part A)

```
python run.py index --dir "path/to/images" --limit 1000 --batch 16
```

Runs YOLO5-Tiny + CLIP on all images. ~0.8 s/image on CPU.

Step 3: Launch Web Dashboard

```
python -m streamlit run app.py
```

Opens browser at localhost:8501. Sidebar has benchmark query presets.

Step 4: CLI Search (Part B)

```
python run.py search "A person in a bright yellow raincoat" --show
```

Prints ranked table and opens top result with bounding-box overlay.

Step 5: Run Evaluation Suite

```
python test_queries.py
```

Evaluates all 5 benchmark prompts and writes annotated JPEG grids.